

Tema 3

Mihai Cristian-Laurentiu 2B4, Stamate Valentin 2B4

03.01.2021

Problema 1

a. Vom incerca sa demonstram prin inductie in functie de k ca exista un cuplaj de acelasi cardinal k care expune cel putin un nod pendent al lui T .

Pentru $k = 1$: T nu are un cuplaj perfect deci $|T| \geq 3$, deci exista doua cuplaje de cardinal 1, oricare dintre ele expune un nod pendent.

Pentru $k = 2$: T nu are un cuplaj perfect deci $|T| \geq 5$, T are $|T| - 1$ arce, astfel daca M_1 satisface toate nodurile pendante, M_2 , format din tot doua muchii (una comuna cu M_1 si o alta muchie care nu exista in M_1) pe cazul minimal cu 5 noduri, nu va satisface nodurile pendante satisfacute de M_1 .

Pentru $k = n$: T nu are un cuplaj perfect deci $|T| \geq 2n + 1$, T are $|T| - 1$ (deoarece este arbore) arce si cuplajul are cardinal n , deci daca un cuplaj arbitrar M va satisface toate nodurile pendante, exista (cel putin) cuplajul M' care poate contine $n-1$ muchii din M si o muchie $e \notin M$ care nu satisface un nod pendent. Astfel in cuplajul M' de acelasi cardinal cu M expune un nod pendent.

Astfel, oricare are fi $k = |M|$ exista un alt cuplaj de acelasi cardinal care expune un nod pendent.

b. Presupunem ca T are un cuplaj perfect M . Daca M este cuplaj perfect (are cardinal maxim) atunci acesta satureaza toate nodurile din T (in consecinta si toate nodurile pendante). Astfel, orice cuplaj de acelasi cardinal va satisface toate nodurile(pendante) din T deoarece are acelasi cardinal cu M care este cuplaj perfect.

In mod evident, daca exista un cuplaj maximal care nu satureaza toate nodurile pendante, acesta nu satureaza toate nodurile din T (nu este cuplaj perfect). Deci un alt cuplaj de acelasi cardinal va expune cel putin un nod pendent (rezulta din a.)

Astfel, T are un cuplaj perfect daca si numai daca orice cuplaj maximal satureaza toate nodurile pendante

Problema 2

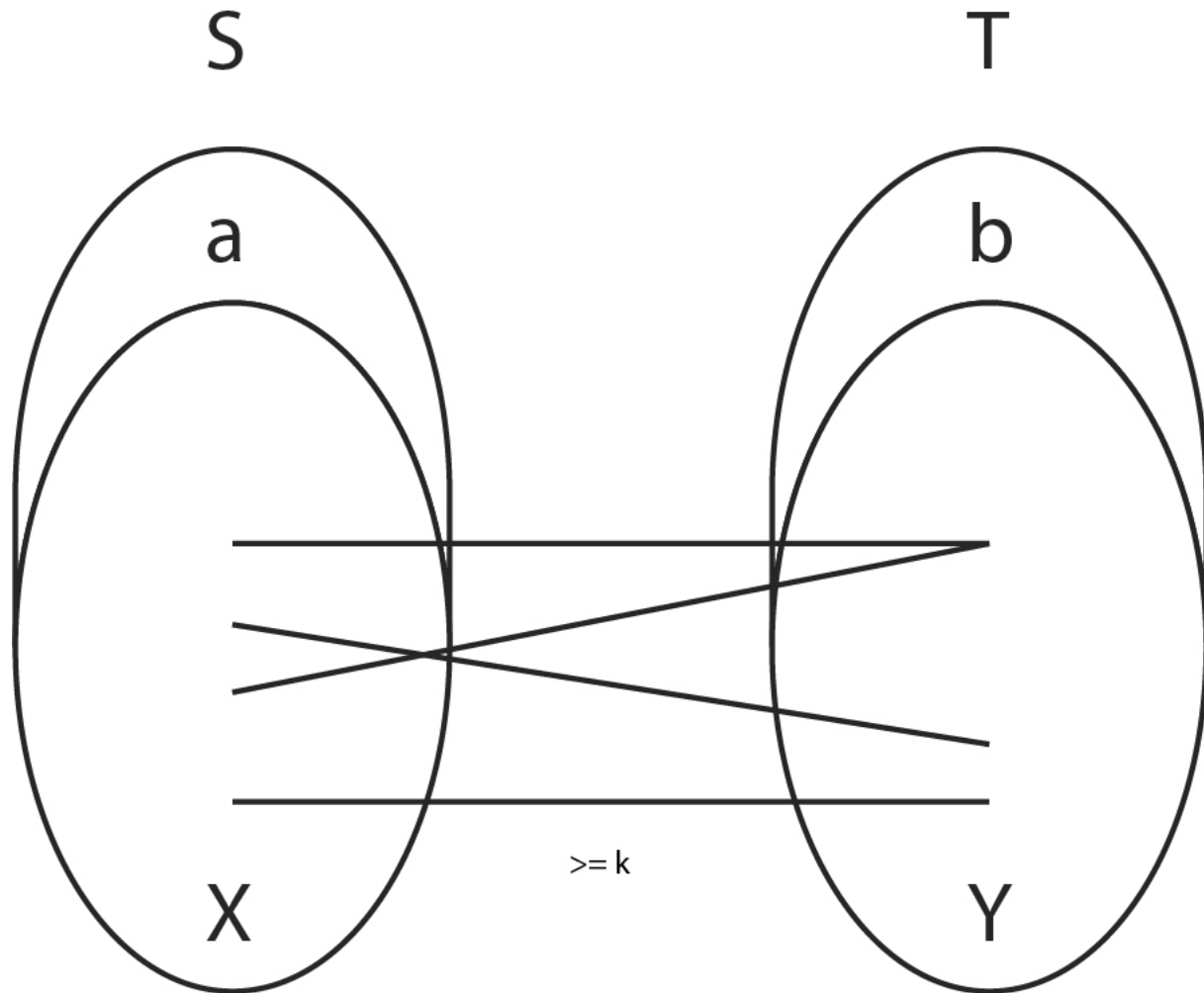
Algoritmul incepe prin apelul $\text{Flow}(s, C_0)$, si pe masura ce se parcurg vecinii (in adancime) apelurile sunt $\text{Flow}(v, \min(c[uv], \alpha))$, astfel observam ca apelurile functiei sunt realizate fie cu capacitatea curenta a arcului fie cu capacitatea ramasa din apelurile facute de stramosii nodului curent. In ceea ce priveste fluxul, acesta se completeaza dupa ce parcurgerea (pe o ramura) ajunge la nodul t , de exemplu arcul $[xt]$ cu capacitatea c va avea fluxul $\min(c, \alpha)$. Apoi in ordine inversa parcurgerii, functia returneaza valoarea maxima a fluxului pentru fiecare arc (in cazul nodurilor din care se creeaza o ramificatie, adica are cel putin 2 copii, valoarea maxima va fi suma valorilor fluxurilor din arcele catre copii). Se observa destul de usor ca frunzele care nu au arc catre t vor avea fluxul 0. Astfel daca la fiecare a apel al functiei Flow se returneaza valoarea maxima a unui flux, tabloul x va fi un flux maxim.

Algoritmul prezentat este asemanator cu o parcurgere DFS a retelei, in sensul ca pentru a putea determina fluxul maxim, se viziteaza vecinii nodului curent (in adancime), astfel putem spune ca algoritmul are complexitatea $\mathcal{O}(n + m)$, unde $n=|V|$ si $m=|E|$.

Problema 3

a. Stim ca $|X| = r$, unde $1 \leq r < n$. Cum $S = X \cup \{x_1', x_2', \dots, x_{n-r}'\}$, $x_i \notin W$ atunci $|S| = |X| + n - r = r + n - r = n$, deci $|S| = n$. T fiind egal cu $V \setminus S$ atunci $|T| = |V| - |S| = 2|W| - n = 2n - n = n$, deci $|T| = n$. Astfel, am demonstrat ca $|S| = |T| = n$.

Pentru a doua parte de demonstrat avem urmatoarea imagine:



unde $a = \{x_1', x_2' \dots x_{n-r}'\}$ iar $b = W' \setminus a$. Stim ca muchiile care conecteaza S si T fac parte din multimea F . Eliminam muchiile intre S si T , astfel nu va mai exista nici o muchie care sa le conecteze. Mergand mai departe, unind toate celelalte moduri exceptand perichile ce fac parte din multimea F vom obtine exact multimea de muchii E . Deoarece $|S| = |T| = n$, bipartitia (S, T) va fi unita prin exact $n^2 - k$ deoarece n^2 vine de la unirea tuturor nodurilor x, y cu $x \neq y, x \in S, y \in T$ adica $n \cdot n = n^2$ si scadem k deoarece initial avem k muchii care conecteaza S si T dar care sunt eliminate. Astfel, am demonstrat ca modulul multimii de muchii care uneste S si T este $\leq n^2 - k$. Q.E.D

b. Ne putem folosi de subpunctul a deoarece constructia grafului corespunde exact cu ipoteza de la acest subpunct. Astfel, stim deja ca multimea de noduri din X care face parte din S este nevida datorata conditiei de la subpunctul a : $1 \leq |X| < n$. Analog pentru Y deoarece T este obtinut din $V \setminus S$, astfel cel putin un nod din W se va afla in T . Atunci X si Y sunt nevide. Deja cunostem faptul ca, avand desenul initial, sunt cel putin k muchii care conecteaza S si T ,

adica o multime de muchii care apartine subgrafului indus de W . Atunci a doua afirmatie este adevarata, adica ceea ce aveam de demonstrat.

c. Mai intai trebuie sa demonstram ca $MBC \in NP - complete$, dupa care sa reducem $simple - MAX - CUT$ la MBC in timp polinomial. Aratand cele 2 lucruri putem spune ca MBC este o problema $NP - complete$.

Algoritmul de mai jos rezolva problema MBC:

```

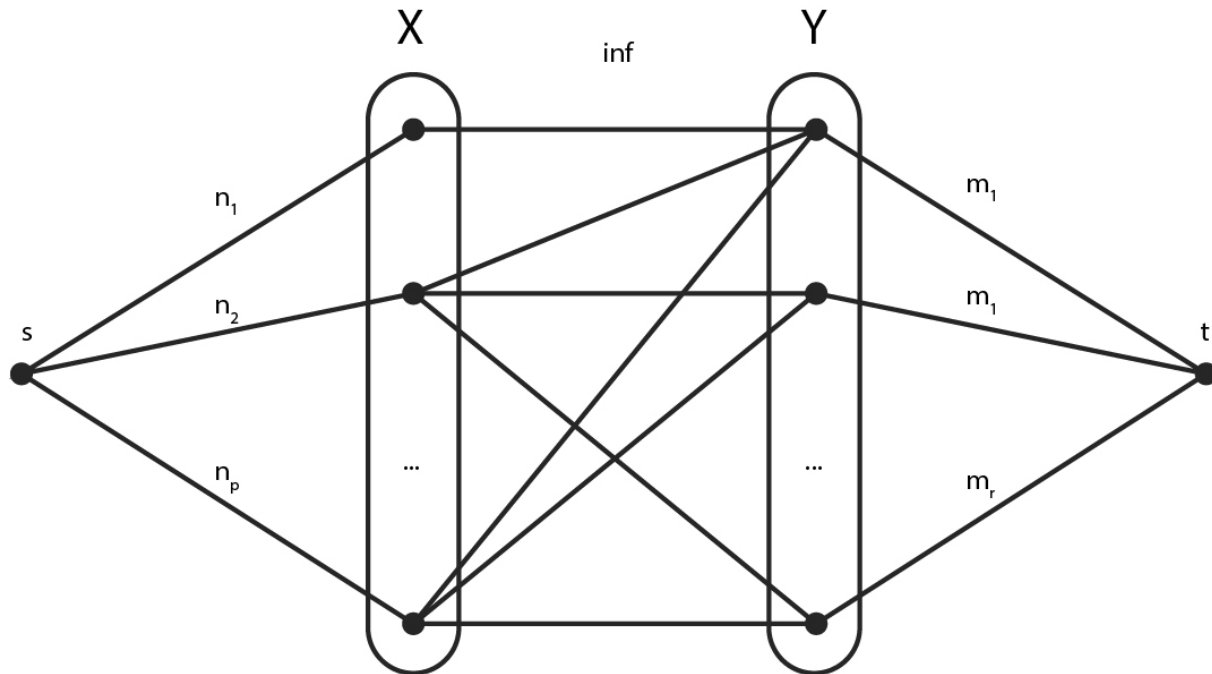
readGraph( $G, E, k$ )
 $n = |G|$ 
 $S = ()$ 
 $nodeList = [1..n]$ 
for  $i = 1; i \leq n/2; i++$  do
     $node = nodeList[rand(1, nodeList.length)]$ 
     $nodeList.remove(node)$ 
     $S.push(node)$ 
end for
 $T = nodeList$ 
 $m = 0$ 
for  $i \in S$  do
    for  $j \in T$  do
        if  $(i, j) \in E$  then
             $m++$ 
        end if
    end for
end for
if  $m \geq k$  then
    print("Exista o solutie")
end if

```

Acest algoritm nedeterminist rezolva problema in timp polinomial deci ea se afla in $NP - complete$.

Problema 4

a. Un model bazat pe o retea de transport pentru a rezolva problema poate fi descris cu un graf bipartit astfel: In prima partitie X vom avea multimea claselor iar in a doua partitie Y noua multime de clase asa cum se poate vedea in imaginea urmatoare:



Atasam doua noduri S adjacent in mod direct cu toate nodurile din X , si T care este adjacent cu nodurile din multimea Y . Numarul de elevi din fiecare grupa poate fi reprezentat ca si un cost pe muchia dintre nodul S si clasa curenta adica $\forall x \in X \ c_{sx} = cost(x)$ unde $cost(x)$ reprezinta numarul maxim de elevi din grupa x . Numarul de elevi al grupelor din multimea Y poate fi asociat in mod similar, costurile fiind puse pe muchiile dintre nodurile din multimea Y si nodul T . Subgraful asociat multimii $S \cup T$ este un $K_{p,r}$ deoarece vrem ca fiecare elev din grupa $x \in X$ sa poata fi redistribuit la orice grupa din $y \in Y$. Costul este infinit deoarece suntem oricum limitati de costurile dintre multimea Y si t , astfel o noua grupa nu poate contine mai mult de c studenti din grupa veche. Astfel am obtinut o retea de transport.

b. Determinarea unei solutii a acestei probleme se poate rezolva aplicand un algoritm de flux maxim. O clasa oarecare poate ceda oricati studenti incap in noua clasa. Atunci cand raman studenti, ei vor fi repartizati intr-o alta clasa. Muchiile dintre cele doua multimii ne pot ajuta sa identific acest lucru astfel: $\forall x, y \ x \in X \ si \ y \in Y$ - "Clasa x a cedat f_{xy} studenti clasei y ", unde f_{xy} reprezinta fluxul dintre nodurile x si y . Aceasta modelare are solutie daca si numai daca suma consturilor asociate multimii X este egala cu suma fluxurilor asociate multimii Y deoarece, ca sa reorganizam toti studentii in alte grupe, vom avea nevoie de acelasi numar de studenti (ca suma a studentilor noilor grupe) in noua organizare.

c. Pentru a determina si verifica existenta unui flux maxim pentru a acesta problema avem urmatorii algoritmi:

- Algoritmul lui Ford & Fulkerson cu etichetarea nodurilor folosind parcurgerea bfs are complexitatea $\mathcal{O}(n \cdot |E|^2)$. Pentru verificarea existentei solutiei la final verificam daca fluxul maxim este egal cu suma capacitatilor asociate partitiei X .
- Un algoritm bazat pe cel precedent este cel al lui Edmonds & Karp ce foloseste parcurgerea bfs cu o complexitate de $\mathcal{O}(n \cdot |E|^2)$. Pentru verificarea solutiei se procedeaza la fel.
- Deoarece avem un graf bipartit putem gasi un algoritm mai eficient ce ruleaza in $\mathcal{O}(p \cdot r)$ ce este descris mai jos.

```

students  $\leftarrow 0$ 
for  $i \in X$  do
     $f(s, i) = c(s, i)$ 
     $students+ = c(s, i)$ 
end for
for  $i \in X$  do
    for  $j \in Y$  do
         $st \leftarrow f(s, i)$ 
         $dr \leftarrow c(j, t) - f(j, t)$ 
         $dr+ = st$ 
         $st = 0$ 
         $f(s, i) \leftarrow s$ 
         $f(j, t) \leftarrow d$ 
         $b \leftarrow d - c(j, t)$ 
        if  $b \leq 0$  then
            continue
        end if
         $f(s, i)+ = b$ 
         $f(j, t) \leftarrow c(j, t)$ 
    end for
end for
     $fstudents \leftarrow 0$ 
for  $j \in Y$  do
     $fstudents+ = f(s, i)$ 
end for
if  $fstudents = students$  then
    print(" Noua organizare este posibila")
else
    print(" Noua organizare nu este posibila")
end if

```